WEEK 2 · LESSON 9 OF 24 · HANDOUT

# Cleaning Messy Data

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=iYie42M1ZyU — open the lesson online to watch inside the course.

**Your goal in this lesson:** Find and fix the most common data problems — missing values, duplicates, wrong types, messy text and impossible values — and document what you changed.

## Real data is always messy

Data typed by people, exported from systems or joined from several sources is never perfect. Data scientists often spend more time **cleaning** data than modelling it. A model trained on dirty data learns dirty patterns.

## A messy dataset

Run this cell. It builds a small file the way it might arrive from a shop's records:

```
import numpy as np
import pandas as pd

raw = pd.DataFrame({
    "date": ["2026-03-01", "2026-03-01", "2026-03-02", "2026-03-02", "2026-03-03",
             "2026-03-03", "2026-03-04", "not recorded", "2026-03-05", "2026-03-05"],
    "district": ["Gasabo", "gasabo ", "Kicukiro", "KICUKIRO", "Nyarugenge",
                 "Gasabo", None, "Kicukiro", "Nyarugenge", "Nyarugenge"],
    "product": ["Rice 5kg", "Rice 5kg", "Milk 1L", "Sugar 1kg", "Milk 1L",
                "Airtime 1000", "Milk 1L", "Rice 5kg", "Sugar 1kg", "Sugar 1kg"],
    "quantity": ["4", "4", "6", "two", "3", "5", "2", "1", "-3", "2"],
    "unit_price": [7500, 7500, 800, 1600, 800, 1000, 800, 7500, 1600, None],
})
print(raw)
```

## Step 1 — Inspect

```
raw.info()
print(raw.isna().sum())          # missing values per column
print(raw["district"].unique())  # spelling variants
```

## Step 2 — Remove exact duplicates

```
print(raw.duplicated().sum())    # 0: the copies only match after cleaning
df = raw.copy()
df["district"] = df["district"].str.strip().str.title()   # fix text first
df = df.drop_duplicates()
print(len(raw), "->", len(df))
```

Clean the text **before** dropping duplicates: `"Gasabo"` and `"gasabo "` only look the same after cleaning.

## Step 3 — Fix types

```
df["quantity"] = pd.to_numeric(df["quantity"], errors="coerce")
df["date"] = pd.to_datetime(df["date"], errors="coerce")
df.info()
```

`errors="coerce"` turns values that cannot be converted (`"two"`, `"not recorded"`) into **missing** (`NaN` / `NaT`) instead of crashing. Now you can decide what to do with them.

## Step 4 — Impossible values

```
print(df[df["quantity"] <= 0])
df.loc[df["quantity"] <= 0, "quantity"] = np.nan    # a sale cannot be negative
```

Ask: *is this value possible in the real world?* Negative quantities, ages of 340, or prices of 0 for paid items are errors.

## Step 5 — Handle missing values

There is no single right answer. Options:

| Option | Code | When it makes sense |
|---|---|---|
| Drop the rows | `df.dropna(subset=["quantity"])` | Few rows affected, and the value is essential |
| Fill with a fixed value | `df["district"].fillna("Unknown")` | Categories, where "unknown" is honest |
| Fill from known facts | look up the product's price | The correct value can be recovered |
| Fill with mean/median | `df["x"].fillna(df["x"].median())` | Numbers, when few are missing — use with care |

```
prices = {"Rice 5kg": 7500, "Milk 1L": 800, "Sugar 1kg": 1600, "Airtime 1000": 1000}
df["unit_price"] = df["unit_price"].fillna(df["product"].map(prices))
df["district"] = df["district"].fillna("Unknown")
df = df.dropna(subset=["quantity", "date"])
df["quantity"] = df["quantity"].astype(int)
df["total"] = df["quantity"] * df["unit_price"]
print(df)
```

## Step 6 — Write a cleaning log

Professionals record every change, so others can check and repeat it:

```
- Standardised district names (strip spaces, Title Case): 2 values changed
- Removed 1 exact duplicate row
- quantity "two" and date "not recorded" could not be read -> rows dropped (2)
- quantity -3 is impossible -> row dropped (1)
- 1 missing unit_price filled from the official price list
- 1 missing district set to "Unknown"
```

## Key terms

- **Missing value (NaN)** — an empty cell; `isna()` finds them.
- **Duplicate** — a row that appears more than once.
- **Type conversion** — e.g. `pd.to_numeric`, `pd.to_datetime`.
- **coerce** — turn unreadable values into missing instead of failing.

- **Imputation** — filling missing values with an estimate.
- **Cleaning log** — a written record of every cleaning step.

**Remember this:** Inspect, standardise text, remove duplicates, fix types, remove impossible values, handle missing values thoughtfully — and write down every step.

Name: _____ Date: _____

## Activity

### Task 1

Run all the steps and check that the final table has no missing values (`df.isna().sum()`).

### Task 2

Change one decision: instead of dropping the row with quantity `"two"`, set it to 2. How does the total income change?

### Task 3

Write your own cleaning log for your version, in plain sentences.

## Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Why clean district text before removing duplicates?

   A. Duplicates cannot be removed otherwise
   B. It makes the file smaller
   C. Rows like 'Gasabo' and 'gasabo ' only match after cleaning
   D. Pandas requires it

2. What does `pd.to_numeric(col, errors="coerce")` do with the text 'two'?

A. Crashes the program
B. Deletes the column
C. Converts it to 2
D. Turns it into a missing value (NaN)

3. A sale has quantity -3. What should you do?

A. Treat it as an impossible value: fix it if possible, otherwise remove it
B. Multiply all quantities by -1
C. Keep it — all data is correct
D. Ignore the whole dataset

4. Which method counts missing values in each column?

A. df.isna().sum()
B. df.missing()
C. df.count_na()
D. df.describe()

5. When is filling a missing category with 'Unknown' a good choice?

A. Only for dates
B. Always for numbers
C. When the true value cannot be recovered and honesty matters
D. Never

6. Why write a cleaning log?

A. To make the notebook longer
B. To hide mistakes
C. So others can check and repeat every change made to the data
D. Because Pandas needs it

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.